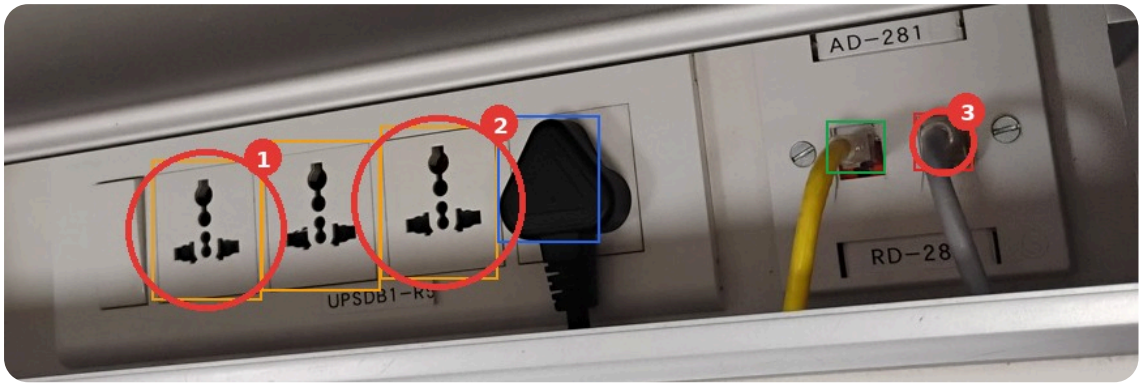
On-Device AI Install Checker — YOLO engine

Same app, same checklist — a second, much smaller AI engine: YOLO object detection + fixed rules.



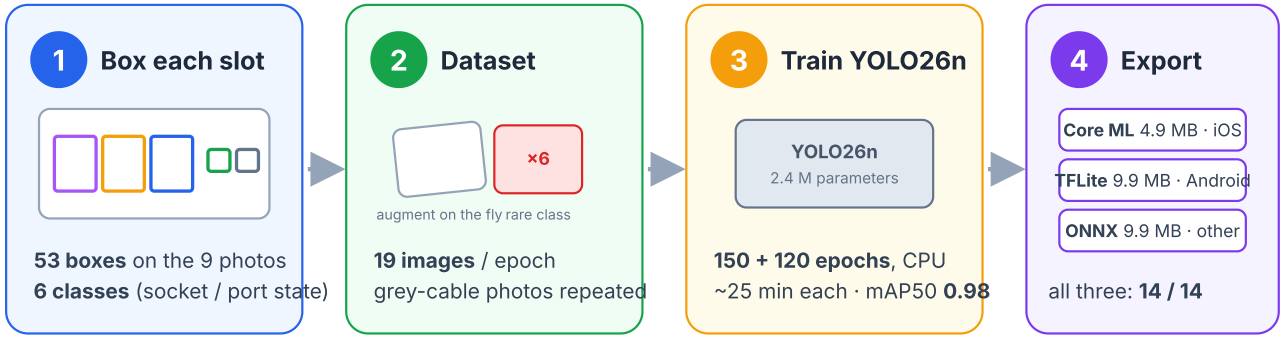
- Model: **YOLO26n** (Ultralytics), trained on our 9 photos
- **5 MB** instead of 3.86 GB — downloads in seconds
- Finds **14 / 14** problems on the demo photos (Gemma: 12 / 14)
- One model, **iOS (Core ML)** and **Android (TFLite)**
- Fully offline · switch engines in the app's *About* screen

1 · Why a second engine?

	YOLO26n + rules	**Gemma 4 E2B (LoRA)**
What the AI does	finds each socket / port and its state	reads the whole photo and writes the answer
Who writes the explanation	fixed rules → always the same wording	the model itself
Model size on phone	**4.9 MB** (Core ML) · 9.9 MB (TFLite)	3.86 GB
Issues found on the 9 demo photos	**14 / 14**, 0 false alarms	12 / 14
Training	~50 min on a laptop CPU	~2.5 h on MacBook GPU + 10 min convert
Android	ready-made `.tflite`	same `.litertlm` (LiteRT-LM)
Best for	fixed checklists, speed, accuracy of the circle	free-form questions, new kinds of problems

Both engines output the same JSON, `{"status","errors":[{x,y,r,desc}]}`, so the app draws the circles the same way.

2 · How we trained it — Step 1 · 2 · 3 · 4



Round 1 found 12/14 issues (missed the grey cable, only 2 examples) → repeat those photos ×6 → round 2: 14/14

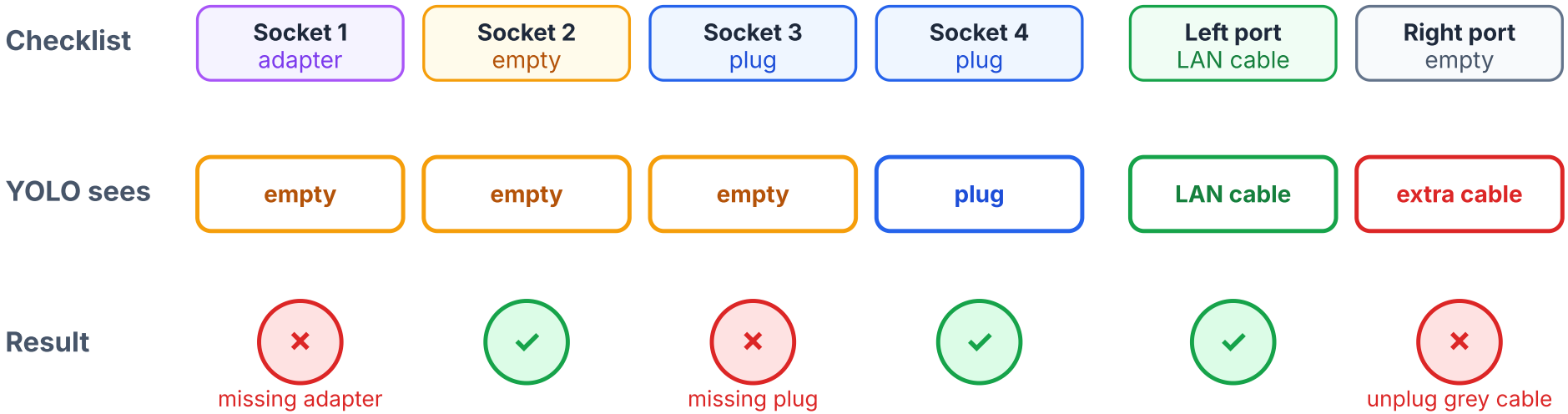
Example of one label (step 1) — instead of circling problems, we draw a box around **every** socket and port and name its state:



Class	Meaning
adapter	power adapter plugged in
plug	3-pin power cord plugged in
empty	empty power socket
LAN	yellow LAN cable in a data port
extra	any other cable in a data port
port	empty data port

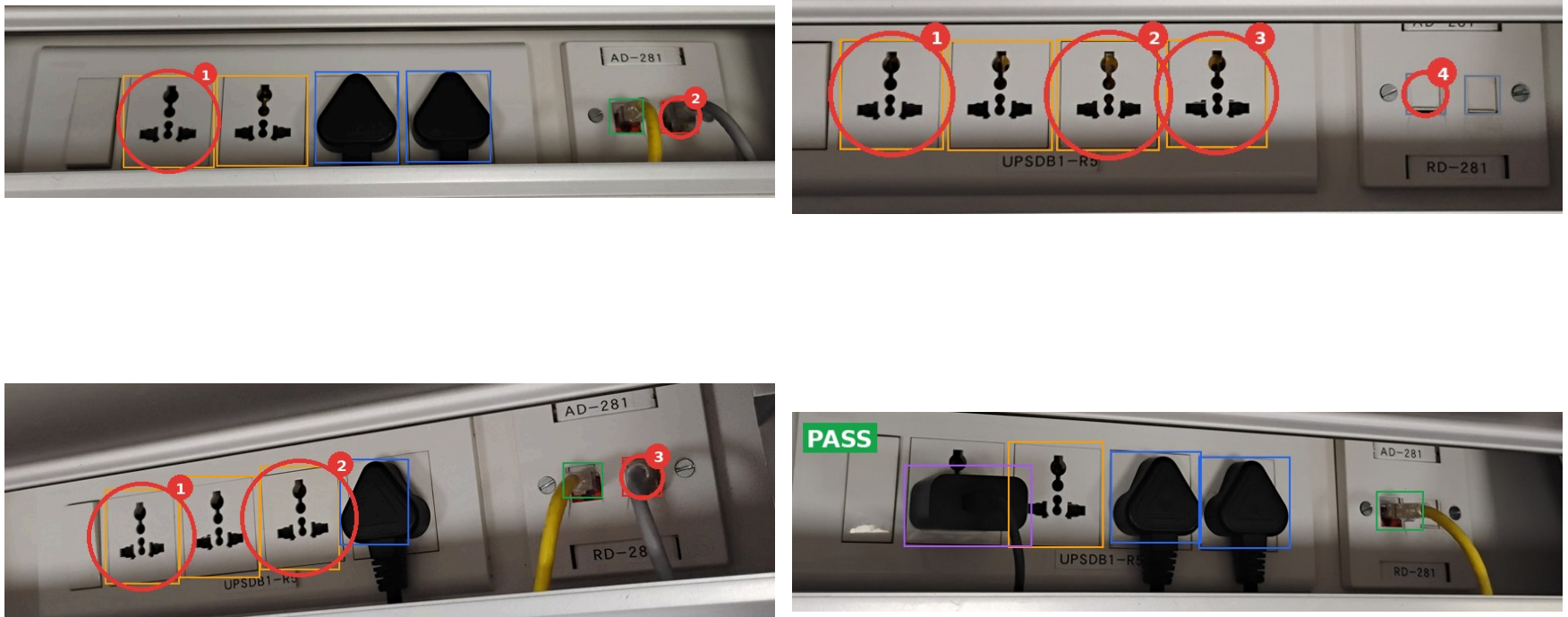
3 · From boxes to problems: the checklist rules

YOLO only says *what is in each slot*. A few lines of code compare that with the standard setup.



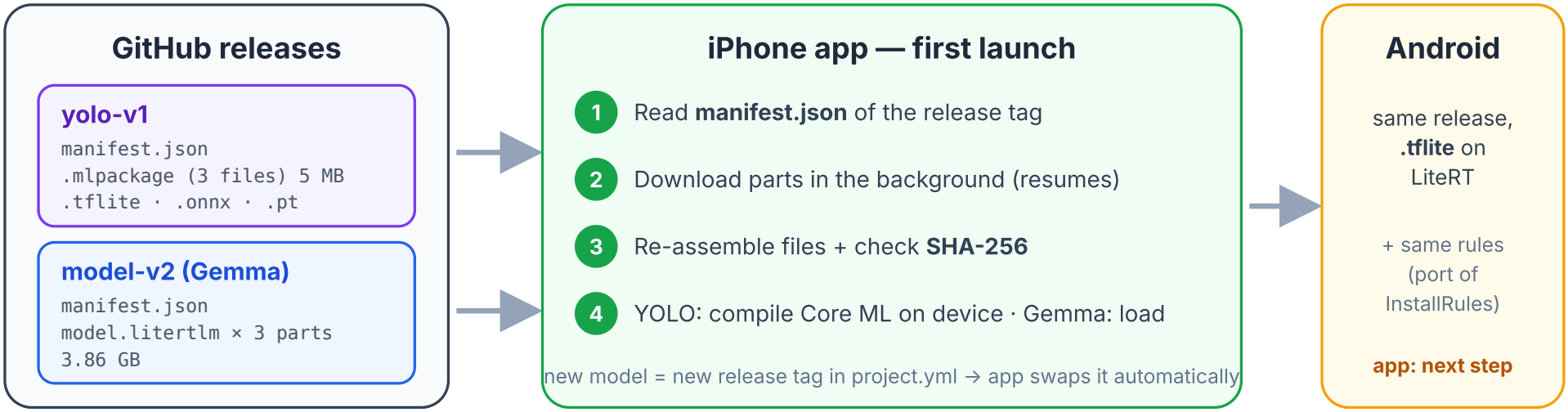
- Sockets are sorted **left → right** and must be: adapter · empty · plug · plug. The data ports need **one yellow LAN cable** and **no other cable**.
- Every mismatch becomes one red circle around that slot, plus a fixed fix-it sentence, e.g. *"Missing power plug. Plug a power cord into the third socket."*
- The rules live in `InstallRules.swift` (app) and `yolo/rules.py` (training repo). Changing the checklist means changing the rules, not retraining.

4 · Input → Output



Thin boxes = what YOLO detected · red circles = problems found by the rules · bottom-right: a correct install → PASS.

5 · Shipping the models: GitHub releases



- Models are **not** inside the app. Each model is a GitHub release with a `manifest.json` (files, sizes, SHA-256): `yolo-v1` and `model-v2`.
- A new model is a new release tag; changing the tag in `project.yml` makes every phone download it once.

6 · Numbers

	Result
PASS / FAIL correct	**9 / 9** photos
Problems found	**14 / 14**, 0 false alarms (Core ML is the same network as the tested PyTorch / ONNX / TFLite builds)
Detection quality (mAP50)	**0.98** on the demo photos
Model file	Core ML **4.9 MB** · TFLite **9.9 MB** · ONNX 9.9 MB
Training	2 rounds × ~25 min on a laptop CPU, 2.4 M parameters

Honest limits today: the same as the Gemma engine. There are only 9 photos of one desk, so the model has memorized this demo and won't generalize yet. YOLO also only knows the 6 states it was taught, and anything new needs a new class plus a rule. The Core ML build has not yet been timed on the iPhone.

7 · Next steps

- **Real data:** 30–50 photos per state from technicians (different rooms, lighting, angles), boxed with the same 6 classes, retrained with the same 4 steps (about an hour).
- **Measure on device:** time per photo and accuracy on iPhone 15 Pro with the Core ML build.
- **Android app:** load `NetworkChecker.tflite` with LiteRT and port the ~60 lines of `InstallRules`.
- **Combine engines (later):** YOLO for fast, exact circles + Gemma for free-form explanations of unusual problems.
- **License:** Ultralytics YOLO is AGPL-3.0, which is fine for this POC. A product needs an Ultralytics Enterprise License, or a switch to Apple Create ML (iOS only).